



Práctica 2

ESTUDIO DE ALGORITMOS DE APRENDIZAJE POR REFUERZO

Por:

Víctor Ramírez Arimaha, Marcel Alabart Benoit y Adrià Cebrián Ruiz

Facultad Informática de Barcelona (FIB)

Sistemas Inteligentes Distribuidos (SID)

Índice

1. Descripción	3
1.1. Entorno	3
1.2. Método de Recopilación de los datos	4
1.2.1. Recolección de resultados del algoritmo	5
1.2.2. Análisis de Importancia de Hiperparámetros mediante <i>Random Forest</i>	5
1.2.3. Representación de los datos	6
2. Iteración de valor	7
2.1. Descripción del algoritmo	7
2.1.1. Parámetros	8
2.1.2. Pasos del algoritmo	9
2.2. Análisis de resultados	9
2.2.1. Impacto de las recompensas sobre el rendimiento del agente	9
2.2.2. Tasa de entrenamientos exitosos en función de distintos parámetros	10
2.2.3. Tiempo de entrenamiento en función de γ y ϵ	10
2.2.4. Rendimiento en función del tiempo de entrenamiento, γ y ϵ	11
2.2.5. Comparación entre políticas específicas	12
3. Model Based	15
3.1. Funcionamiento del Algoritmo	15
3.1.1. Parámetros	15
3.1.2. El algoritmo por partes	15
3.2. Estudio	16
3.2.1. Toma de muestras	16
3.2.2. Resultados	16
4. <i>Q-Learning</i>	21
4.1. Descripción del Algoritmo	21
4.1.1. Pasos del Algoritmo	21
4.1.2. Hiperparámetros	23
4.2. Estudio Experimental y Análisis	24
4.2.1. Muestreo de Hiperparámetros	24

4.2.2. Resultados Globales y Análisis de Hiperparámetros	25
5. REINFORCE	32
5.1. Funcionamiento del Algoritmo	32
5.1.1. Parámetros	32
5.1.2. El algoritmo por partes	33
5.2. Estudio	34
5.2.1. Toma de muestras	34
5.2.2. Resultados	34
5.3. Intento de mejora	45
6. Bibliografía	47

1. Descripción

En esta práctica se tratará de estudiar los efectos de cada hiperparámetro y sus respectivas influencias en una selección de algoritmos de aprendizaje por refuerzo. Mediante una serie de experimentos se buscarán las mejores combinaciones de hiperparámetros para los algoritmos de aprendizaje por refuerzo seleccionados, en términos de tiempo de ejecución y calidad de los resultados.

1.1. Entorno

El entorno en el que se estudiará es *CliffWalking-v0*, un entorno de *Gymnasium*.

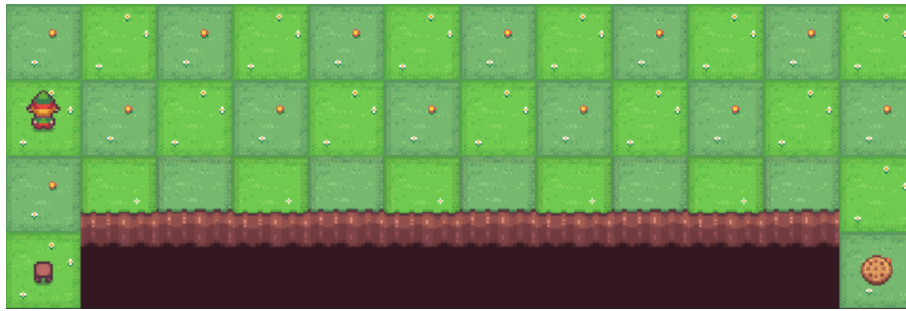


Figura 1: Entorno *CliffWalking-v0*

Como se puede observar, el entorno es una matriz de 4x12. El estado inicial es la esquina inferior izquierda, que se corresponde al estado 36. El estado final es la esquina inferior derecha de la matriz, que se corresponde al estado 47.

- **Acciones:** El entorno dispone de 4 acciones, estas son: moverse una posición arriba, a la derecha, abajo o a la izquierda. Cada una de estas acciones tiene un 1/3 de probabilidades de mover al agente en la dirección seleccionada. En caso de no moverse en la dirección seleccionada el agente se desplazará una unidad hacia cualquier dirección excepto la opuesta a la seleccionada. Al haber solo dos posibles direcciones no opuestas que no se traten de la seleccionada, cada una tiene una probabilidad equivalente de 1/3. Por ejemplo, si el agente seleccionase la acción ".arriba", se desplazaría con 1/3 de probabilidades hacia arriba, con 1/3 hacia la izquierda y con 1/3 hacia la derecha, sumando 1 en total. No cabe duda de que el entorno es completamente estocástico. En caso de un intento de desplazamiento hacia una dirección fuera de las posiciones de la matriz el agente no se desplazará. Por ejemplo, al desplazarse a la izquierda en la posición inicial, el agente permanecerá en la posición inicial.

- **Recompensas por defecto:** Cada vez que el agente se desplaza a una posición que no sea un barranco (posiciones [37, 38,..., 46]), este recibe una recompensa de -1, si cae a un barranco la recompensa es de -100. Al alcanzar el estado final (47) la recompensa recibida es de 0. Estas recompensas serán modificadas en los experimentos para estudiar los efectos que pueden tener sobre la eficacia de los distintos algoritmos que se verán en este documento.
- **Barrancos:** Como se ha mencionado en el punto anterior, los barrancos se sitúan en las posiciones [37, 38,..., 46], si el agente cae en uno de ellos recibe una recompensa de -100. Sin embargo, el juego no termina, en cambio el agente se mueve automáticamente al estado inicial en la posición 36.

1.2. Método de Recopilación de los datos

Es evidente que existen varias influencias entre los hiperparámetros. Por ejemplo en caso de *Q-Learning*, ϵ_{decay} se aplica en cada episodio por lo que una mayor cantidad de episodios de entrenamiento implicará un mayor decrecimiento de ϵ . Es decir que, para analizar su efecto sobre la calidad de la política entrenada de forma correcta, se debería realizar un *grid search* completo, ya que no conocemos todas las influencias entre hiperparámetros de antemano ni podemos garantizar su independencia. En otras palabras *grid search* es el análisis combinatorio de todos los valores discretos que se hayan decidido estudiar para cada hiperparámetro. Esto ha demostrado ser inviable computacionalmente debido a la *curse of dimensionality*, la enorme cantidad de posibles combinaciones dispara exponencialmente el coste del análisis. El estudio del algoritmo *Q-Learning*, por ejemplo, hubiese tomado 14 días en ejecutarse.

Por obvias razones, esto es inalcanzable, de manera que lo que se realizó un diseño *random search*. Este método reduce el número de experimentos al evaluar solo una fracción de las combinaciones posibles, manteniendo la capacidad de identificar efectos significativos de los hiperparámetros y sus interacciones clave. Como indica su nombre se trata de escoger un cierto número de combinaciones al azar dentro del espacio de hiperparámetros, en este caso se han escogido alrededor de un 10% de las combinaciones posibles. Cómo se indica en *Bergstra & Bengio 2012* [1] esta cantidad puede ser suficiente en la mayoría de los casos, sobre todo cuando un par de hiperparámetros predominan sobre el impacto en el rendimiento.

Este metodo fué utilizado para todas las recolecciones de muestras excepto para el

algoritmo de *Value Iteration*, debido a que este es un algoritmo muy simple y la simulación de muestras en *grid search* se puede llevar a cabo en un tiempo aceptable con los hiperparámetros escogidos. Además al tener un ^{en}entrenamiento completamente determinista no son necesarias varias muestras para indicar el rango de error de los datos (ya que no hay error).

En el resto de algoritmos se han recolectado cinco muestras por combinación, ya que un mayor número implicaría el sacrificio de cantidad de combinaciones aleatorias seleccionadas.

Es importante destacar que una implementación OAT (*One At a Time*), como las que realizamos en la práctica de búsqueda local en la asignatura de inteligencia artificial del grado de ingeniería informática, no captaría las influencias entre hiperparámetros.

Resumidamente: por todas las razones mencionadas previamente y el deseo de aprender nuevas técnicas de análisis y optimización se han implementado *grid search* y *random search*.

1.2.1. Recolección de resultados del algoritmo

Como se menciona previamente, todos los algoritmos salvo *Value Iteration* serán analizados con 5 muestras (entrenamientos) por combinación aleatoria de hiperparámetros.

La implementación en *Python* mayoritariamente hace uso de la librería *Sklearn* para elegir las configuraciones aleatorias de hiperparámetros. Debido a diferencias en los miembros del equipo, también se ha utilizado otro método: generar un archivo .json con todas las combinaciones que se deben analizar mediante elección aleatoria simplemente utilizando la librería *random*. Cada combinación y sus resultados fueron almacenados en archivos de tipo .csv para, más adelante, llevar a cabo representación y análisis de los datos.

1.2.2. Análisis de Importancia de Hiperparámetros mediante *Random Forest*

Para cuantificar la influencia relativa de cada hiperparámetro en el rendimiento del agente (específicamente en cantidad de pasos hasta el objetivo en caso de alcanzarlo), se entrenó un modelo de regresión *Random Forest*. *Random Forest* es un método de aprendizaje conjunto (*ensemble learning*) que construye múltiples árboles de decisión durante el entrenamiento y produce la media de las predicciones de los árboles individuales para tareas de regresión. Una de sus ventajas es la capacidad de estimar la importancia de cada característica (hiperparámetro en nuestro caso) basándose en cuánto contribuye cada una a reducir la impureza o el error de predicción a través de los árboles. Se utilizó

la puntuación OOB (*Out-Of-Bag*) para evaluar la capacidad de generalización del modelo.

1.2.3. Representación de los datos

Para los algoritmos de *REINFORCE* y *Basado en Modelo*, se han elaborado diagramas de caja (*BoxPlots*) con el objetivo de representar visualmente sus parámetros y facilitar su posterior análisis individual.

No obstante, esta metodología no resulta adecuada para el caso de *Q-learning*, dado el mayor número de hiperparámetros que este algoritmo involucra. Por tal motivo, se optó por utilizar gráficos de líneas con bandas de error (que representan el error estándar), permitiendo así almacenar los datos en una única gráfica manteniendo una interpretación comprensible de los resultados.

Para generar los gráficos se utilizaron *scripts* de *Python* con *Pandas*, *Matplotlib* y *Seaborn*. Estas herramientas permitieron hacer una representación clara y elegante de los datos recolectados.

Además, con el objetivo de practicar y aprender sobre estas librerías, se han incluido otros tipos de gráficos como gráficos de barras apiladas, *heat maps*, *scatter point plots* y *horizontal bar graphs*.

2. Iteración de valor

Esta parte del estudio analiza el funcionamiento del algoritmo de iteración de valor. El método de iteración de valor consiste en encontrar la política óptima estimando los valores óptimos de cada estado-acción.

Nos encontramos con un entorno totalmente observable, donde además sabemos la probabilidad exacta de ir a parar a un estado u otro con una acción determinada. Podemos garantizar que el algoritmo converge debido a que, en las pruebas realizadas, siempre utilizamos un factor de descuento estrictamente menor a 1.

2.1. Descripción del algoritmo

El algoritmo de iteración de valor sigue los siguientes pasos:

1. Inicialización:

- Asignar a todos los estados no finales un valor inicial igual a 0
- Asignar a todos los estados finales un valor inicial igual a su recompensa inmediata

2. Iteración:

- Para cada estado no final $s \in S$:
 - a) Almacenar el valor anterior: $v \leftarrow V(s)$
 - b) Calcular el nuevo valor:

$$V(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$

- b)
 - c) Actualizar Δ : $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

3. Criterio de parada:

- Repetir el paso de iteración hasta que $\Delta < \epsilon$

4. Extracción de la política óptima:

- Para cada estado $s \in S$:

$$\pi(s) \leftarrow \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$

2.1.1. Parámetros

En esta sección se detallan los parámetros del algoritmo de iteración de valor, explicando su propósito y relevancia.

1. Gamma (γ):

- Factor de descuento ($0 < \gamma < 1$)
- Determina la importancia de recompensas futuras frente a las inmediatas
- Valores cercanos a 1 hacen que el agente planifique a largo plazo
- Esencial para garantizar la convergencia del algoritmo
- En nuestras pruebas usamos los valores [0.9, 0.95, 0.99]

2. Epsilon (ϵ):

- Umbral de convergencia (valor pequeño > 0)
- Determina cuándo detener las iteraciones (cuando $max_diff < \epsilon$)
- Valores más pequeños producen políticas más precisas pero requieren más iteraciones
- En nuestro caso usamos $\epsilon = [1, 0.01, 0.0001, 0.000001]$

3. Recompensas:

- **Recompensa por finalización (R_{final}):**
 - Recompensa inmediata al alcanzar el estado objetivo
 - En nuestras pruebas usamos [0, 10, 100]
- **Penalización por paso (R_{paso}):**
 - Penalización por acción
 - En nuestras pruebas usamos [0, -1, -10]
- **Penalización por caer (R_{caer}):**
 - Penalización por entrar en estados no deseados

- En nuestras pruebas usamos $[-10, -100, -1000]$

4. Max_iterations:

- Límite superior de iteraciones como medida de seguridad
- Utilizamos un valor de 1000, en ningún experimento hemos llegado a este umbral antes de la convergencia

2.1.2. Pasos del algoritmo

2.2. Análisis de resultados

En este estudio evaluamos cómo varían las diferentes métricas de rendimiento del agente, en concreto los pasos de media necesarios para llegar al final y el tiempo de entrenamiento del agente, según los parámetros explicados anteriormente.

2.2.1. Impacto de las recompensas sobre el rendimiento del agente

Como se aprecia en la figura 2, a cuanto menor recompensa de finalización, más pasos en promedio necesita el agente para alcanzar la meta, mientras que la recompensa por caer no tiene demasiado impacto sobre los pasos promedios. En la figura 3 filtramos las ejecuciones que llegan a T-MAX pasos, es decir, las ejecuciones que no llegan a alcanzar la meta en, en este caso, 200 pasos. Para una recompensa de finalización de 100 no hay ninguna diferencia significativa entre las figuras. Para una recompensa de 10 ya se nota una ligera diferencia y, para una recompensa de 0, vemos que la diferencia es sustancial, cosa que indica que la tasa de ejecuciones fallidas aumentan a medida que disminuye la recompensa de finalización.

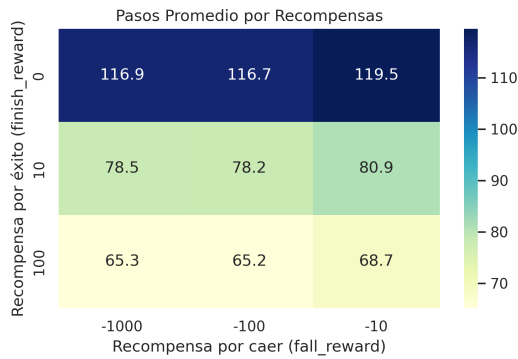


Figura 2: Pasos promedio en función de las recompensas (todas las ejecuciones)

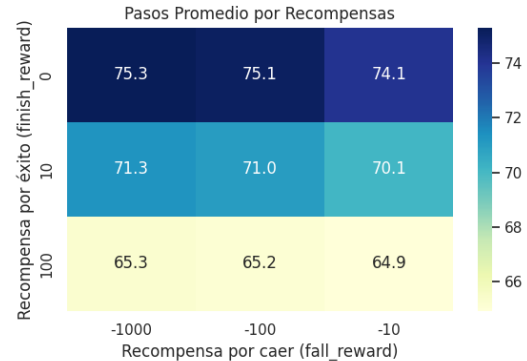


Figura 3: Pasos promedio en función de las recompensas (ejecuciones exitosas)

2.2.2. Tasa de entrenamientos exitosos en función de distintos parámetros

Como se puede apreciar en la figura 4, el numero de configuraciones fallidas aumenta a medida que disminuimos el factor de aprendizaje γ y, dentro de una γ más baja, las configuraciones con un valor de convergencia ϵ más alto obtienen más fallos en comparación a valores más bajos. Este comportamiento es el esperado debido a que la política resultante del algoritmo de iteración de valor no tiene tiempo a estabilizarse con unos valores de convergencia altos y un factor de descuento más penalizante.

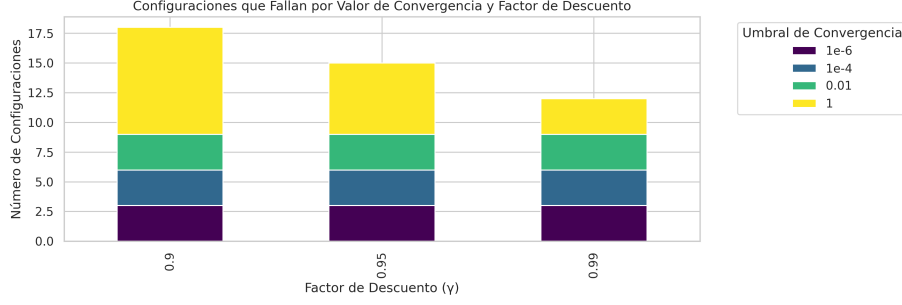


Figura 4: Configuraciones fallidas en base a γ y ϵ

2.2.3. Tiempo de entrenamiento en función de γ y ϵ

Como es de esperar, el tiempo de entrenamiento del agente aumenta tanto en base al valor de convergencia como al valor del factor de descuento. Como se aprecia en la figura 5, a medida que disminuye el valor de convergencia, para los mismos valores de γ el tiempo de entrenamiento aumenta. En el gráfico vemos también que todos los valores de convergencia tienen al menos una configuración fallida, pero se ve que la media de pasos en una convergencia de 1 es mayor que en el resto, donde exceptuando las configuraciones fallidas todas tienen una media de pasos menor o igual a 75.

Aunque se ve una tendencia de aumento en el tiempo de entrenamiento, este aumento es mínimo cuando hablamos de valores, siendo la mayor diferencia de unos 0.05 segundos.

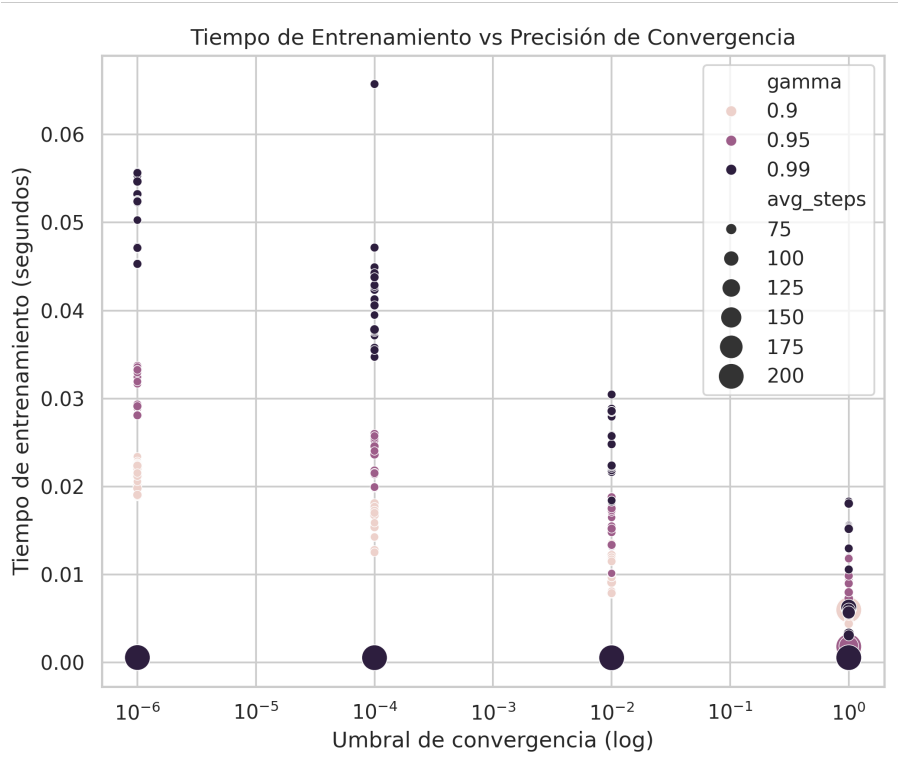


Figura 5: Tiempo de entrenamiento en base a γ y ϵ

2.2.4. Rendimiento en función del tiempo de entrenamiento, γ y ϵ

Como se aprecia en la figura 6, el tiempo de entrenamiento no tiene un impacto significativo en la media de pasos de un política. A partir de unos 10ms, todas las políticas tienen una media de pasos similar.

En la figura podemos ver como, a medida que γ se aproxima a 1, obtenemos mejores resultados, ϵ sigue el mismo comportamiento, a medida que disminuye, mejores resultados obtenemos.

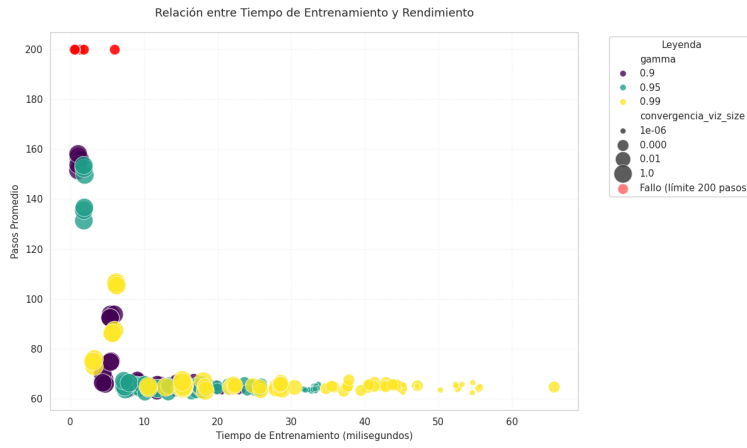


Figura 6: Pasos de media respecto al tiempo de entrenamiento

2.2.5. Comparación entre políticas específicas

Para la mejor política encontrada se han utilizado los siguientes parámetros:

ϵ : 0.01

γ : 0.99

Recompensa finalización: 10

Recompensa por caerse: -1000

Penalización por paso: -10

Pasos de media: 62.21

Se puede ver que, a excepción de las dos casillas superiores de la primera columna, el camino parece ser el más directo posible que no permite caerse al vacío. Estas dos casillas pueden diferir de lo que parece el mejor camino debido a que tal vez el valor de convergencia es demasiado alto. También cabe la posibilidad de que entre las pruebas haya una combinación mejor de parámetros pero que la media de pasos, al ser un entorno estocástico, haya salido un poco peor pese haber utilizado 500 episodios para calcularla.

```

Learned Policy:
[['^' '>' '>' '>' '>' '>' '>' '>' '>' '>' '>']
['^' '>' '>' '>' '>' '>' '>' '>' '>' '>' '>']
['^' '^' '^' '^' '^' '^' '^' '^' '^' '^' '^']
['<' '^' '^' '^' '^' '^' '^' '^' '^' '^' '^']]

```

Figura 7: Mejor política

Para una de las peores políticas sin ejecuciones exitosas se probaron los siguientes parámetros:

ϵ : 1
 γ : 0.9
 Recompensa finalización: 0
 Recompensa por caerse: -10
 Penalización por paso: 0
 Pasos de media: 200.0

Se puede ver en la figura 8 que no existe la posibilidad de llegar a la meta debido a que, en la esquina superior derecha, el agente no puede bajar hacia la última casilla, quedándose para siempre haciendo movimientos en la fila superior. Al tener un factor de descuento bajo y un valor de convergencia alto, el algoritmo hace pocas iteraciones antes de considerar que ya es estable, por lo que el 0 de recompensa por finalizar no llega a expandirse demasiado.

```

Learned Policy:
[['^' '^' '^' '^' '^' '^' '^' '^' '^' '^' '^' '^']
 ['<' '^' '^' '^' '^' '^' '^' '^' '^' '^' '^' '^>']
 ['<' '^' '^' '^' '^' '^' '^' '^' '^' '^' '^' '^>']
 ['<' '^' '^' '^' '^' '^' '^' '^' '^' '^' '^' '^']]
  
```

Figura 8: Peor política

Para la peor política con ejecuciones exitosas se probaron los siguientes parámetros:

ϵ : 1
 γ : 0.9
 Recompensa finalización: 10
 Recompensa por caerse: -1000
 Penalización por paso: -1
 Pasos de media: 158.06

Se puede ver en la figura 9 que existe la posibilidad de llegar a la meta pero, a diferencia de la mejor política encontrada, el camino no es directo y depende de que falle en todos los pasos hacia la dirección correcta, haciendo que haya solo un 33 % de probabilidad en cada paso de ir hacia la dirección correcta. Al igual que en el resto de políticas comentadas, no es posible que el agente caiga al vacío.

Las diferencias entre parámetros con la mejor política encontrada son el valor del factor de descuento, el valor de convergencia y el valor de la penalización por paso. Es posible que al tener un factor de descuento menor, la importancia de la recompensa final se propaga cada

vez con menos importancia entre iteraciones, haciendo que como los huecos de vacío están más cerca del jugador, estos tengan mucha más importancia y el agente toma una ruta más conservadora, incluso si sigue siendo imposible caerse. El valor de convergencia puede haber influido a la hora de hacer que la política se considere estable cuando aún podría cambiar en iteraciones futuras. El impacto de la penalización por paso puede haber influido en el sentido de que el agente considera que es mejor tardar más, al ser una penalización menor, que jugársela a caer al vacío.

```

Learned Policy:
[[['^', '^', '^', '^', '^', '^', '^', '^', '^', '^', '^', '^'],
  ['<', '<', '^', '^', '^', '^', '^', '^', '^', '^', '^', '^'],
  ['<', '^', '^', '^', '^', '^', '^', '^', '^', '^', '^', '^'],
  ['<', '^', '^', '^', '^', '^', '^', '^', '^', '^', '^', '^']]]

```

Figura 9: Peor política con éxitos

3. Model Based

Para llevar a cabo el estudio se utilizó el método de *Estimación Directa*. Con unos pequeños retoques. A continuación se explica el funcionamiento del algoritmo y el análisis de los hiperparámetros.

3.1. Funcionamiento del Algoritmo

El objetivo principal de este algoritmo es que el agente sea capaz de representar un modelaje aproximado del entorno en el que trabaja. En esta implementación de *Estimación Directa* el agente toma pasos aleatorios para intentar conseguir más muestras con tal de lograr una estimación más precisa y fidedigna al MDP verdadero del entorno en el que trabaja.

3.1.1. Parámetros

Aquí se listarán los parámetros que toma el algoritmo, y se mencionará brevemente para que se usen, se entrará en mas profundidad sobre la verdadera implementación en el siguiente apartado.

1. **Gamma:** Factor de descuento, utilizado para que los calculos de utilidad no sea infinito por la existencia de bucles.
2. **Num_trajectories:** Numero de pasos aleatorios que el agente toma para aprender sobre el entorno en cada iteracion de valor.
3. **Epsilon:** Este parametro ha sido añadido con la intencionalidad de que reemplace el *Reward Threshold* que habia anteriormente. De esta manera se intenta conseguir que el algoritmo de el mejor resultado con los parametros seleccionados. Se decidió que era mejor que coger la primera politica que tuviese una *reward* inferior al *threshold*. Además, el *Reward Threshold* no facilita los estudios con nuevas recompensas, y se decidió que la posible alternativa de utilizar el numero de *Steps* era bastante pobre.

3.1.2. El algoritmo por partes

A continuación listamos los pasos que toma el algoritmo:

1. Tanto el agente como el entorno son inicializados, asignándole a este los parámetros.

2. Bucle de entrenamiento

- a) El agente toma $Num_trajectories$ pasos aleatorios, con tal de aprender sobre el entorno.
- b) El agente ejecuta el algoritmo de iteración de valor con tal de ver si ha convergido.
- c) Si la diferencia maxima entre las utilidades anteriores y las calculadas es menor al $Epsilon$ (parametro añadido), sal del bucle.

3. El entrenamiento se da por terminado.

3.2. Estudio

En este apartado se va a explicar e intentar comprender los resultados dados por la experimentación con los diferentes parámetros.

3.2.1. Toma de muestras

Como ya se ha explicado en anteriormente se tomaran muestras usando un diseño *random search*. Se eligieron los siguientes posibles parámetros:

$$\begin{aligned} StepReward &\in \{-10, -1, 0\}, FinishReward \in \{0, 10, 100\}, \\ FallingReward &\in \{-1000, -100, -10\}, NumberOfTrajectories \in \{1000, 5000, 1000\}, \\ Gamma &\in \{0.9, 0.95, 0.99\}, Epsilon \in \{0.005, 0.01, 0.1, 1.0\} \end{aligned}$$

Existen un total de $3^5 * 4 = 972$ combinaciones posibles de con estos parámetros. usando *random search* decidimos recolectar una muestra de 100 elementos, es decir aproximadamente un 10 % de las combinaciones totales.

3.2.2. Resultados

Como se menciona en el apartado anterior, se consiguieron mas de 100 muestras. Se ejecuto el algoritmo de análisis de importancia de parametros *Random Forest* sobre el numero promedio de pasos. El algoritmo devuelve un *OOB Score* de 0.551 (cosa que implica un alto rango de error en la predicción), y devuelve los siguientes resultados:

- **step_reward:** 0.3738
- **Epsilon:** 0.2642
- **Número de trayectorias:** 0.1224

- **finish_reward:** 0.1055
- **Gamma:** 0.0815
- **fall_reward:** 0.0527

3.2.2.1 Efecto de las recompensas

En este apartado se va a analizar el efecto de los cambios de recompensas en los resultados de los experimentos.

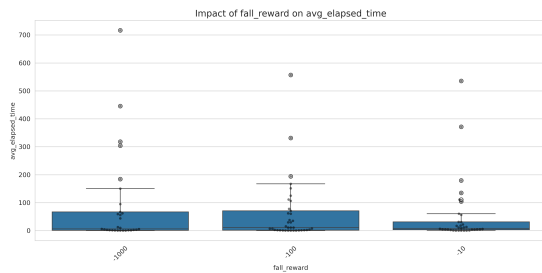


Figura 10: Tiempo de entrenamiento según la recompensa de caída

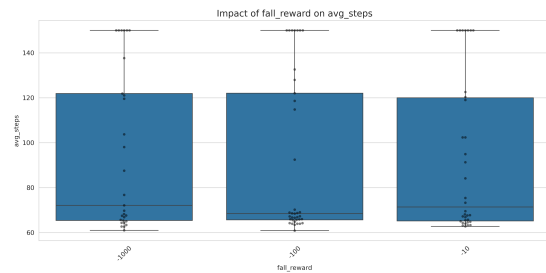


Figura 11: Promedio de pasos según la recompensa de caída

Como se puede distinguir de las figuras 10 y 11, fijando una recompensa por caída de -10, se consigue de media ligeramente mejor de pasos pasos, consumiendo considerablemente menos tiempo en el entrenamiento.

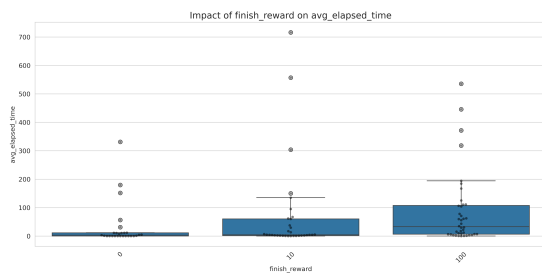


Figura 12: Tiempo de entrenamiento según la recompensa final

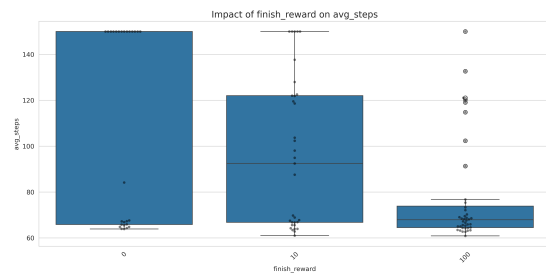


Figura 13: Promedio de pasos según la recompensa final

Los boxplots de las figuras 12 y 13 son muy interesantes. se puede comprobar como cuando se le añade una gran recompensa al estado final (en este caso una recompensa de 100), el tiempo de entrenamiento aumenta considerablemente. Esto seguramente sea debido a que, al haber añadido el parámetro *epsilon* para intentar que la *Iteracion de Valor* tenga una mínima convergencia, un valor tan grande tarda bastante más en expandirse, de ahí el aumento drástico en el tiempo.

Por otro lado, se puede observar la exagerada mejora en el promedio de pasos. De manera que con los parámetros seleccionados se debería encontrar un tipo de equilibrio entre tiempo y calidad de la solución (número de pasos), depende de lo que se priorice más.

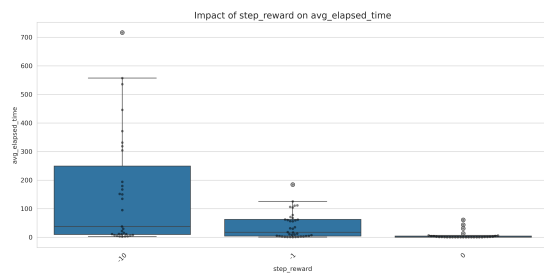


Figura 14: Tiempo de entrenamiento según la recompensa por paso

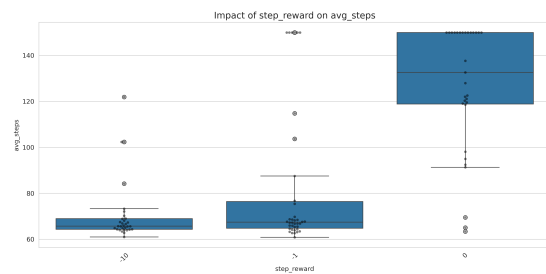


Figura 15: Promedio de pasos según la recompensa por pasos

En este caso, se repite una situación muy similar a la que había con las anteriores figuras 12 y 13. En los *BoxPlots* de las figuras 14 y 15 se puede observar como, a medida que disminuye la recompensa por paso, tarda mucho menos en entrenarse, al igual que en las figuras 12 y 13, si seleccionamos una recompensa por paso de 0, el algoritmo de iteración por valor converge mucho antes, aunque consiguiendo unos pasos promedios terribles. El mejor resultado en número de pasos es una recompensa por paso de -10, pero el mejor equilibrio parece estar en la recompensa original del entorno, -1.

3.2.2.2 Efecto de Gamma

En este apartado se verá como afecta el parámetro gamma al rendimiento del algoritmo.

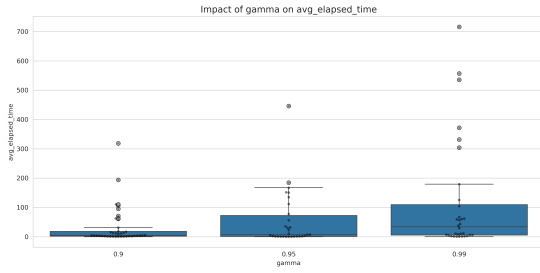


Figura 16: Tiempo de entrenamiento según el parámetro γ

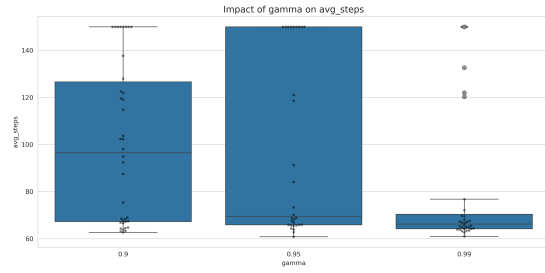


Figura 17: Promedio de pasos según el parámetro γ

Los resultados de las figuras 16 y 17 son los esperados. Con una gamma mayor, al tener que computar como afectan los estados mas lejanos a una utilidad, tarda mucho más tiempo. Sin embargo, al tener utilidades mas fidedignas al MDP original, y tener en cuenta estados lejanos de una forma más efectiva, el valor 0.99 para gamma es el que nos consigue el mejor numero de pasos.

3.2.2.3 Efecto del numero de trayectorias

A continuación se analizara como afecta el parámetro de número de trayectorias al rendimiento del algoritmo.

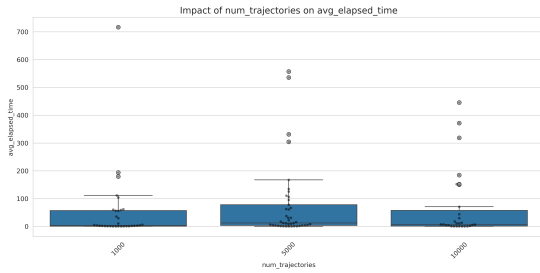


Figura 18: Tiempo de entrenamiento según el número de trayectorias

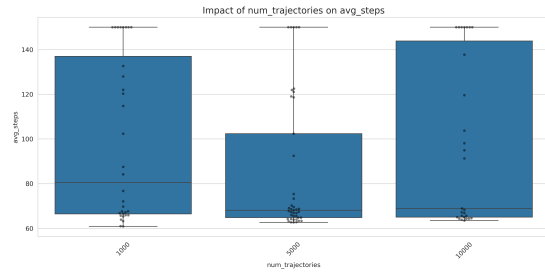


Figura 19: Promedio de pasos según el número de trayectorias

Sorprendentemente, las diferencias de tiempos entre los entre los diferentes valores de este parámetro no es mucha. Además, las ejecuciones con 5000 como numero de trayectoria tardaron más que las que tenían 10000 como numero de trayectorias. Sin embargo, la asignación de 5000 tubo con diferencia los mejores resultados en términos de número de pasos.

3.2.2.4 Efecto del parámetro añadido *Épsilon*

En este apartado se comprobará como afecta el parámetro añadido de convergencia al algoritmo.

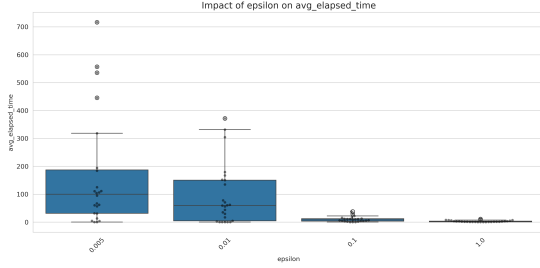


Figura 20: Tiempo de entrenamiento según el parámetro *épsilon*

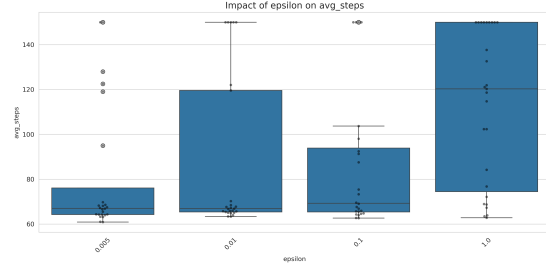


Figura 21: Promedio de pasos según *épsilon*

Como se puede comprobar en los *BoxPlots* de las figuras 20 y 21, este parámetro añade mucho coste temporal a los algoritmos. Sin embargo, al asegurarse de tener una buena convergencia para el iteración del valor, cuando se asignan valores bajos, como 0.005 o 0.01 en este caso, se consiguen valores bastante en lo que respecta al numero de pasos.

3.2.2.5 Conclusión de los experimentos

A continuación, se muestra una figura con los rankings de las diferentes configuraciones de parámetros, dándole la misma importancia al tiempo de entrenamiento y el numero de pasos.

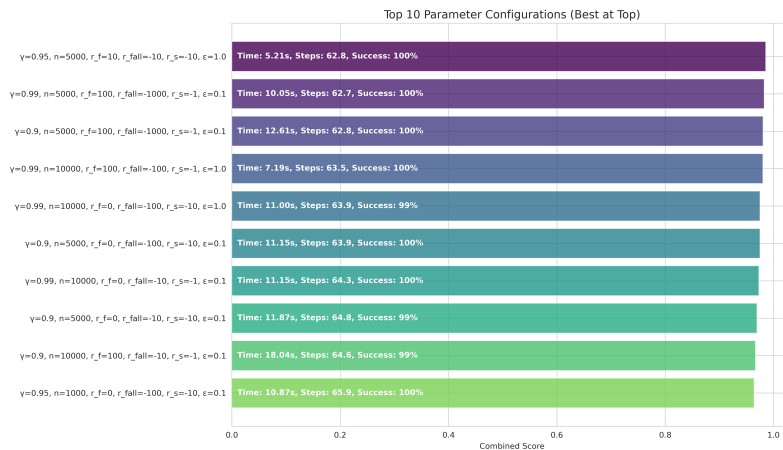


Figura 22: Top 10 configuraciones de parámetros

4. *Q-Learning*

Este estudio se centra en el algoritmo *Q-Learning*, un método de aprendizaje por refuerzo libre de modelo y *off-policy* que aprende una función de valor acción-estado óptima. A diferencia de la Iteración de Valor o métodos basados en modelos, *Q-Learning* no requiere conocer la dinámica completa del MDP (transiciones y recompensas) de antemano, aprendiendo directamente de la experiencia. Su naturaleza *off-policy* le permite aprender de acciones exploratorias mientras estima el valor de la política óptima (*greedy*), lo que lo distingue de algoritmos *on-policy* como Iteración de valor 2 o *SARSA*.

4.1. Descripción del Algoritmo

Q-Learning es un algoritmo de control temporal-diferencial (TD) que busca aprender la función de valor acción-estado óptima, denotada como $Q^*(s, a)$. Esta función representa el retorno esperado máximo al tomar la acción a en el estado s y seguir la política óptima a partir de entonces. El agente aprende iterativamente actualizando sus estimaciones de $Q(s, a)$ basándose en la experiencia recolectada.

4.1.1. Pasos del Algoritmo

El proceso de aprendizaje para *Q-Learning* se puede describir de la siguiente manera para cada episodio:

1. **Inicialización:** Se inicializa la tabla Q , $Q(s, a)$, para todos los pares estado-acción (s, a) con valores arbitrarios (e.g., ceros o pequeños valores aleatorios). Para estados terminales, si $s_{terminal}$ es un estado terminal, $Q(s_{terminal}, a) = 0$ para todas las acciones a . Se establece el valor inicial del coeficiente de exploración ϵ .
2. **Bucle por episodio:** Para cada episodio de entrenamiento:
 - (a) Se inicializa el estado actual s (normalmente el estado inicial del entorno).
 - (b) **Bucle por paso dentro del episodio:** Mientras s no sea un estado terminal y no se alcance el número máximo de pasos por episodio (t_{max}):
 - i. **Selección de Acción:** Se elige una acción a en el estado s utilizando una política derivada de Q , comúnmente ϵ -greedy. Con probabilidad ϵ , se elige una acción aleatoria; con probabilidad $1 - \epsilon$, se elige la acción que maximiza $Q(s, a)$: $a = \arg \max_{a'} Q(s, a')$.

- i. **Ejecución de Acción y Observación:** Se ejecuta la acción a , y el agente observa la recompensa r y el siguiente estado s' . Se determina si s' es un estado terminal (*done*).
- i. **Actualización de Q-valor:** Se actualiza el valor de $Q(s, a)$ utilizando la regla de actualización de Q-Learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Si s' es un estado terminal, el término $\max_{a'} Q(s', a')$ se considera 0.

- i. Se actualiza el estado: $s \leftarrow s'$.
- (c) **Decaimiento de ϵ :** (Opcional pero común) Se reduce el valor de ϵ para disminuir la exploración a medida que el agente aprende.

4.1.2. Hiperparámetros

Los hiperparámetros clave que controlan el comportamiento del algoritmo *Q-Learning* y que se han estudiado son:

- γ (**Gamma - Factor de Descuento**): Valor entre $[0, 1]$. Determina la importancia de las recompensas futuras. Un valor cercano a 0 prioriza recompensas inmediatas, mientras que un valor cercano a 1 da más peso a las recompensas a largo plazo.
- α (**Alpha - Tasa de Aprendizaje**): Valor entre $(0, 1]$. Controla cuánto de la nueva información (el objetivo TD) sobrescribe la información antigua (el valor Q actual). Una tasa alta implica cambios rápidos en Q, una baja implica un aprendizaje más lento y estable.
- ϵ (**Epsilon - Tasa de Exploración Inicial**): Valor entre $[0, 1]$. Probabilidad inicial de elegir una acción aleatoria en lugar de la acción greedy según la estimación actual de Q.
- ϵ_{decay} (**Decaimiento de Epsilon**): Factor por el cual ϵ se reduce después de cada episodio (o paso). Controla la velocidad a la que la política pasa de ser exploratoria a ser explotadora.
- ϵ_{end} (**Epsilon Final**): Valor mínimo al que ϵ puede decaer. Asegura un nivel mínimo de exploración incluso después de mucho entrenamiento.
- **Número de Episodios de Entrenamiento**: Total de partidas que el agente juega para aprender.
- t_{max} (**Pasos Máximos por Episodio**): Límite de pasos en un solo episodio para evitar bucles infinitos o episodios excesivamente largos.
- **Señal de Recompensa (`finish_reward`, `fall_reward`, `step_reward`)**: Valores numéricos que definen la recompensa por alcanzar el objetivo, caer por el precipicio y por cada paso regular, respectivamente. La estructura de estas recompensas es fundamental para guiar el aprendizaje.

4.2. Estudio Experimental y Análisis

El estudio experimental para *Q-Learning* se diseñó para evaluar el impacto de sus hiperparámetros en el rendimiento del agente en el entorno CliffWalking-v0 con la opción `is_slippery=True`. Se utilizó una metodología de muestreo aleatorio de combinaciones de hiperparámetros para explorar el espacio de búsqueda de manera eficiente cómo se explica en 1.2. Cada combinación se ejecutó múltiples veces (5 en este caso) para obtener estimaciones robustas del rendimiento y su variabilidad. El rendimiento se midió principalmente a través del número promedio de pasos para alcanzar el objetivo y la tasa de éxito durante la fase de evaluación de la política entrenada ($\epsilon = 0$). La fase de evaluación realiza 100 episodios después del entrenamiento.

4.2.1. Muestreo de Hiperparámetros

Se definieron rangos y valores discretos para cada hiperparámetro a estudiar. A partir de estos, se generaron 500 combinaciones de hiperparámetros de forma aleatoria para los experimentos. Los rangos considerados fueron:

- γ : {0.9, 0.95, 0.99}
- α : {0.05, 0.1, 0.2, 0.3, 0.5}
- ϵ (inicial): {0.3, 0.5, 0.7, 1.0}
- ϵ_{end} : {0.01, 0.05, 0.1}
- ϵ_{decay} : {0.001, 0.0005, 0.0001, 0.00005}
- **Número de Episodios de Entrenamiento**: {1000, 4000, 8000, 12000}
- **Señal de Recompensa (finish, fall, step)**: {(0, -100, -1), (10, -100, -1), (100, -100, -1), (0, -100, 0)}
- t_{max} : {200}

Cada una de estas combinaciones aleatorias se ejecutó 5 veces y los resultados de evaluación se promediaron, calculando también su desviación estándar reflejada en las bandas de error de los *line plots*. Nótese que existe una combinación de recompensas que **no favorece alcanzar el final** ya que no existe recompensa negativa por *step* ni positiva al llegar al objetivo. En el apartado de *Q-Learning* se ha decidido filtrar los resultados de ésta

combinación en la tasa de finalización. La razón es que se hipotetiza que esta combinación rara vez alcanzará el final y es fundamentalmente diferente en concepto al resto. Sin tener en cuenta esta combinación no hay cambios en la recompensa de caída ni de acción, esto es debido a restricciones en tiempo de computación, un estudio exhaustivo debería tener en cuenta más valores para estas recompensas.

4.2.2. Resultados Globales y Análisis de Hiperparámetros

Tras la ejecución de 500 combinaciones de hiperparámetros, se observó una amplia gama de rendimientos. El análisis de importancia de características mediante un modelo *Random Forest* (OOB Score: 0.8419) indicó que los parámetros más influyentes en el número de pasos en evaluación para episodios exitosos fueron, en orden descendente:

- α : 0.7422
- ϵ_{decay} : 0.0793
- **Número de Episodios de Entrenamiento**: 0.0651
- ϵ (inicial): 0.0398
- ϵ_{end} : 0.0281
- γ : 0.0238
- **finish reward**: 0.0218
- t_{max} : 0
- **step reward**: 0
- **fall reward**: 0

Queda clara la predominancia del hiperparámetro α frente al resto. Esto es justificable por dos razones, α es un coeficiente directo en el aprendizaje del agente por lo que una gran influencia es esperada. Además el rango de valores probados para α es especialmente amplio en comparación al resto, debido a la inexperiencia del equipo con los estándares en hiperparámetros de aprendizaje por refuerzo. Podemos observar, también, que algunos hiperparámetros han resultado tener influencia 0. Ésto se debe a que t_{max} solamente cuenta con un posible valor, por lo tanto no tiene cambio posible. Tanto *step reward* como *fall*

reward no afectan al resultado debido a que su único cambio es en la combinación filtrada (sin incentivo de finalizar). Para realizar más exhaustivamente el estudio se deberían comprobar más posibles valores para estos hiperparámetros.

A continuación, se analiza el efecto individual de los hiperparámetros más relevantes sobre la calidad del aprendizaje, principalmente medida por la eficiencia (pasos hasta el objetivo en evaluación) y la tasa de éxito.

4.2.2.1 Análisis de la Señal de Recompensa (*finish_reward*, *fall_reward*, *step_reward*):

Se observó que la estructura de recompensas tuvo un impacto crítico. Las configuraciones con *finish_reward*=0 y *step_reward*=0 resultaron consistentemente en una tasa de éxito del 0 % en alcanzar el estado objetivo (estado 47), aunque el agente aprendiera a no caerse. Esto subraya la necesidad de una recompensa de finalización positiva y/o un coste por paso para incentivar el comportamiento deseado confirmando la hipótesis del apartado 4.2.2. La configuración estándar (*finish*:0, *fall*:-100, *step*:-1) demostró ser robusta para lograr altas tasas de éxito, permitiendo entonces analizar la eficiencia mediante los pasos.

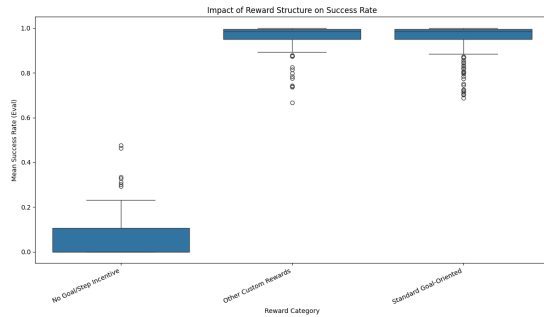


Figura 23: Impacto de la Estructura de Recompensa en la Tasa de Éxito Media de Evaluación.

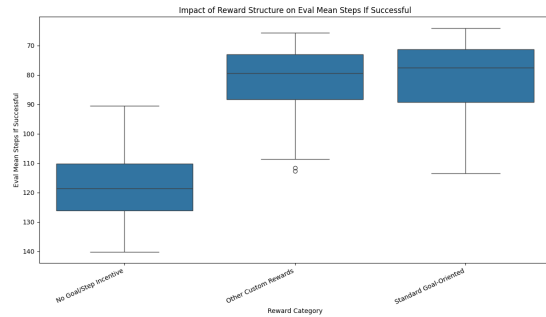


Figura 24: Impacto de la Estructura de Recompensa en los Pasos Medios (si exitoso) de Evaluación.

Se puede observar que en las raras ocasiones en las que la estructura de recompensas que no favorece el alcance del objetivo la cantidad de pasos media hasta el final es mucho mayor al resto. Además se han agrupado los dos tipos de recompensa (finalización con recompensa 10 y 100) en *Other Custom Rewards* debido a sus resultados realmente similares. Se puede observar que el impacto de esta recompensa en la cantidad de pasos hasta el objetivo es realmente mínimo e incluso los valores por defecto los superan ligeramente.

4.2.2.2 Análisis del Decaimiento de Epsilon (epsilon_decay):

Dada la naturaleza estocástica del entorno (`is_slippery=True`), `epsilon_decay` fue uno de los parámetros más críticos. Tasas de decaimiento muy rápidas (0.001) llevaron a un rendimiento subóptimo, ya que el agente dejaba de explorar antes de aprender una política robusta a la aleatoriedad. Tasas más lentas (0.00005) consistentemente produjeron mejores resultados en términos de menor número de pasos para alcanzar el objetivo, aunque requirieron más episodios de entrenamiento para converger. Esto sugiere que una exploración prolongada es esencial en entornos estocásticos.

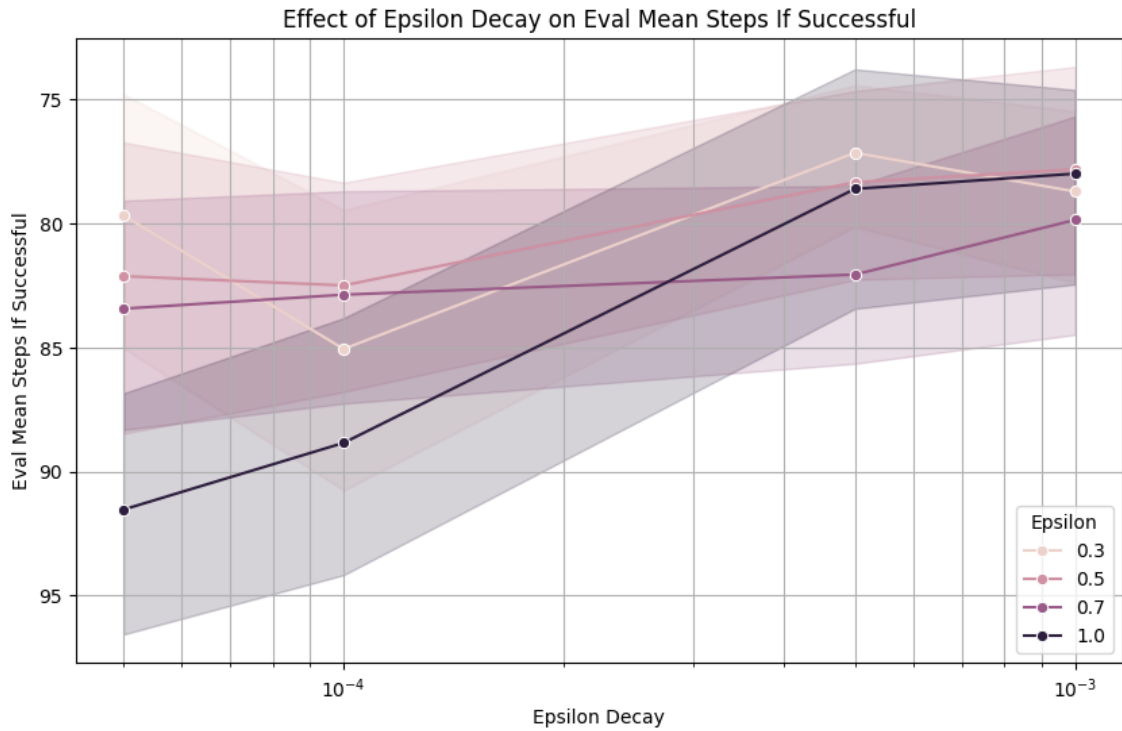


Figura 25: Efecto de ϵ_{decay} (escala log) en los Pasos Medios (si exitoso) de Evaluación, agrupado por ϵ inicial.

4.2.2.3 Análisis de la Tasa de Aprendizaje (alpha):

La tasa de aprendizaje α ha resultado ser el hiperparámetro más influyente en los pasos medios hasta el objetivo. Mostró un comportamiento lineal en el cual los resultados empeoraban según el aumento de α . Se observa que los valores más bajos como 0.05 han obtenido la menor cantidad de pasos hasta el objetivo y además con más consistencia (menor desviación). En caso de buscar una combinación óptima se deberían explorar valores para α aún menores ya que se observa una tendencia lineal descendiente.

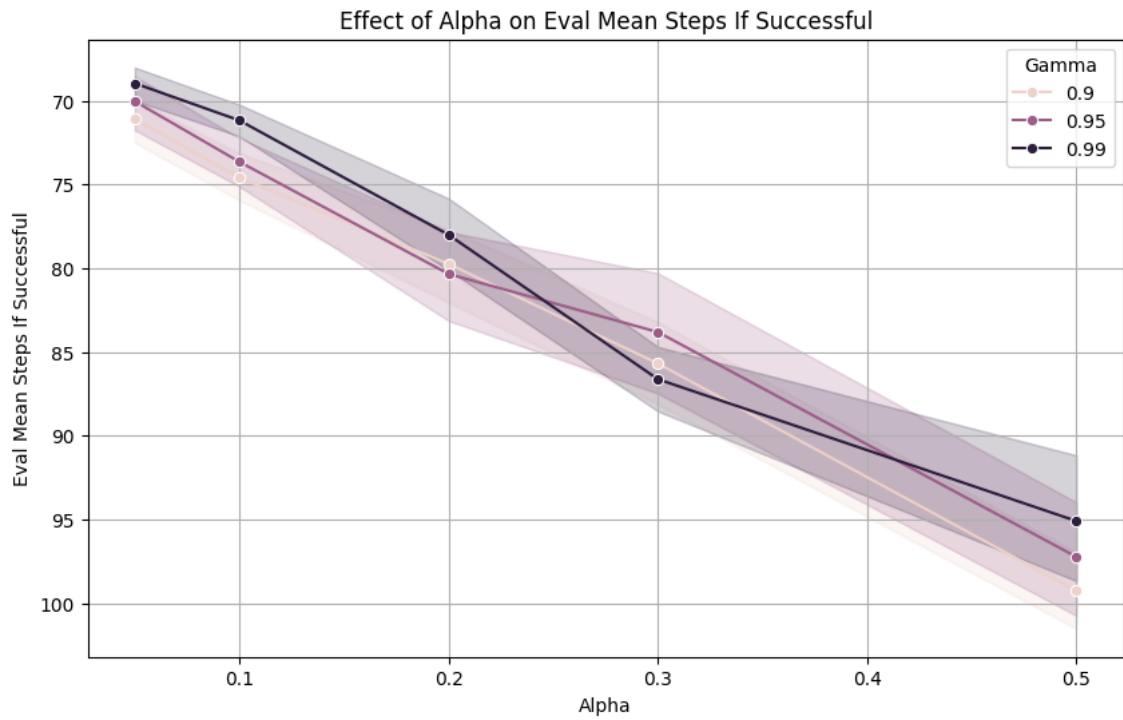


Figura 26: Efecto de α en los Pasos Medios (si exitoso) de Evaluación, agrupado por γ .

4.2.2.4 Análisis del Número de Episodios de Entrenamiento (num_train_episodes):

El número de episodios de entrenamiento mostró un comportamiento no lineal interesante. Se observó que aumentar los episodios de 1000 a 8000 mejoró significativamente el rendimiento (menos pasos). Sin embargo, un incremento adicional a 12000 resultó en un empeoramiento del rendimiento en las muestras recolectadas. Esto podría indicar que, con la configuración de exploración dada, el agente converge a una política (posiblemente subóptima robusta) alrededor de los 8000 episodios, y un entrenamiento adicional no necesariamente refina la política hacia el óptimo global o incluso podría ser perjudicial debido a la continua exposición a la estocasticidad con una exploración mínima.

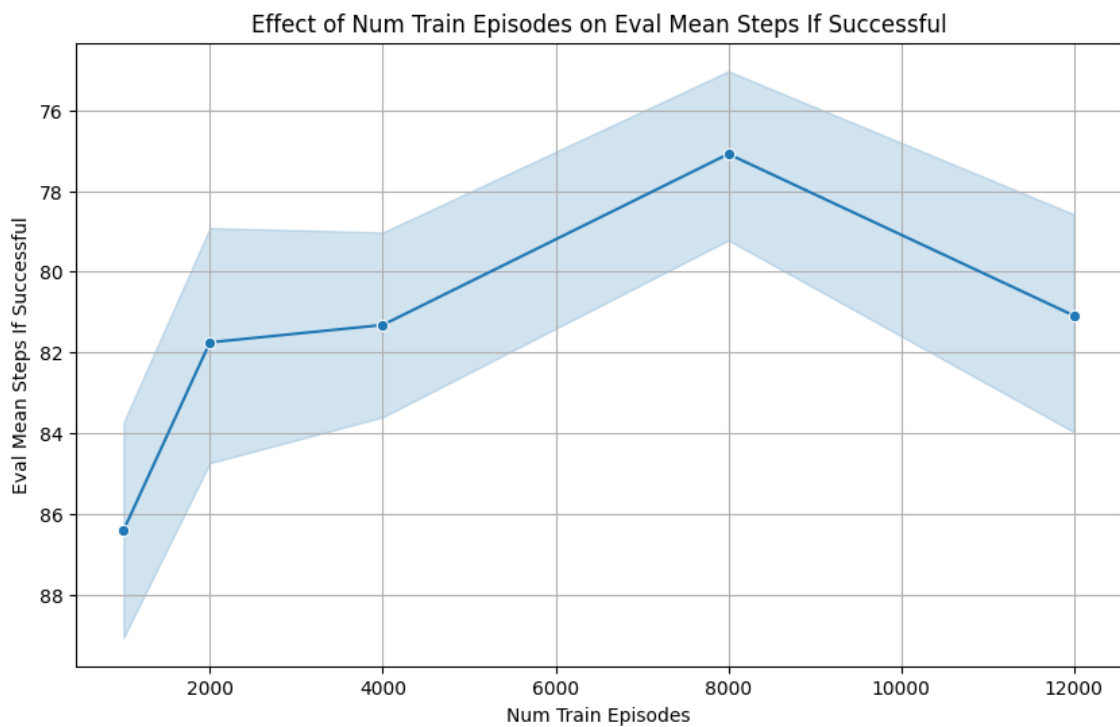


Figura 27: Efecto del Número de Episodios de Entrenamiento en los Pasos Medios (si exitoso) de Evaluación.

4.2.2.5 Análisis de Otros Hiperparámetros (γ , ϵ , ϵ_{end}):

El factor de descuento γ no mostró un impacto tan drástico como otros parámetros dentro del rango probado ($[0.90-0.99]$), aunque valores más altos como 0.99 tendieron a estar presentes en las mejores configuraciones, lo cual es esperable ya que el agente necesita planificar para evitar caídas en un entorno estocástico. Se hipotetiza que **HIPOTETIZAR AQUÍ** Un ϵ inicial más alto (e.g., 0.7 o 1.0) pareció beneficioso cuando se combinó con un decaimiento lento. El valor de ϵ_{end} tuvo un efecto menor, aunque mantener una exploración mínima (e.g., 0.05) podría ser marginalmente mejor que 0.01 en este entorno estocástico para evitar estancarse prematuramente.

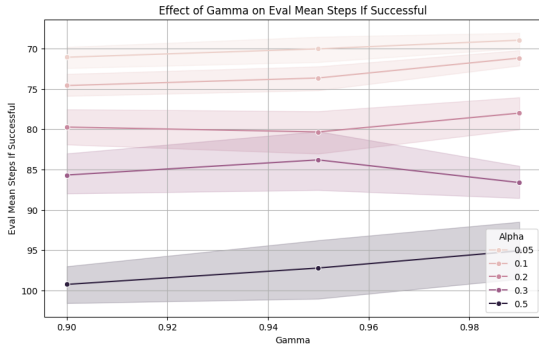


Figura 28: Efecto de γ en los Pasos Medios (si exitoso) de Evaluación, agrupado por ϵ .

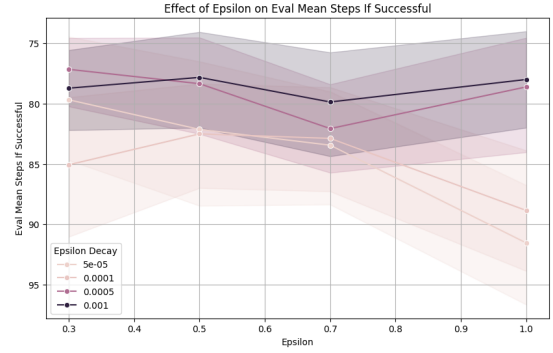


Figura 29: Efecto de ϵ en los Pasos Medios (si exitoso) de Evaluación, agrupado por ϵ_{decay} .

4.2.2.6 Conclusiones del Estudio de Q-Learning y Combinaciones Óptimas (Sugeridas):

El estudio revela que para el entorno CliffWalking-v0 en modo `is_slippery=True`, la gestión de la exploración (especialmente un `epsilon_decay` lento) y una señal de recompensa que incentive claramente el objetivo son primordiales. Una tasa de aprendizaje α baja ha demostrado ser lo más influyente y positivo en el rendimiento del agente. Un número suficiente de episodios (alrededor de 8000) son también importantes. Basado en los resultados, algunas de las combinaciones de hiperparámetros que consistentemente mostraron un alto rendimiento (bajos pasos en evaluación con alta tasa de éxito) incluyen:

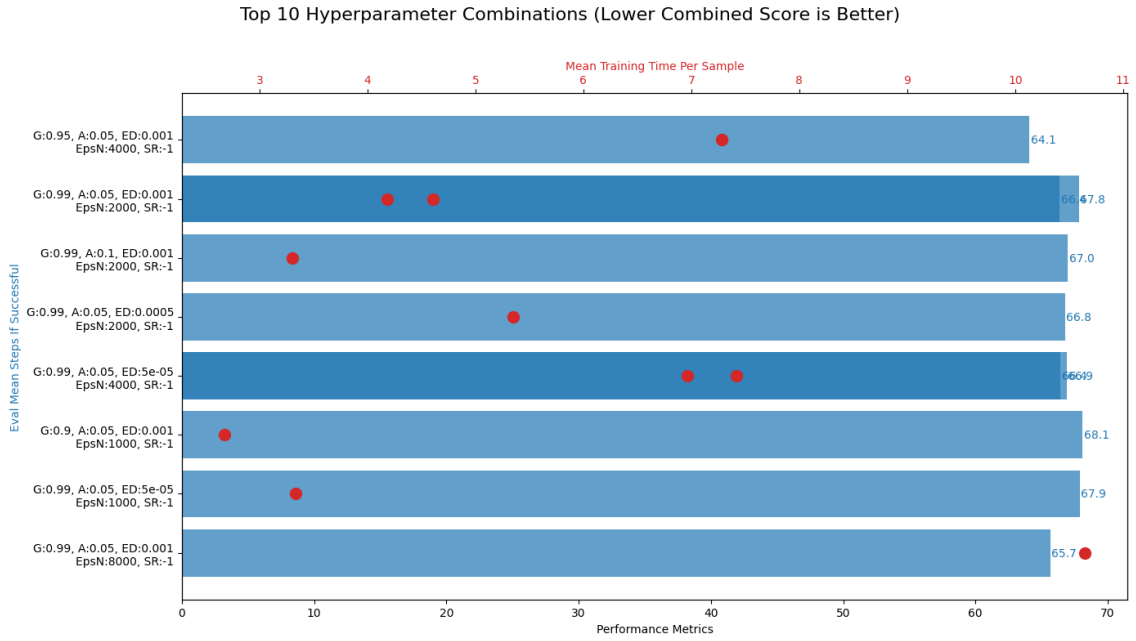


Figura 30: Top 10 combinaciones de hiperparámetros basándose en pasos medios hasta el objetivo y tiempo de ejecución.

La mejor combinación identificada en cuanto a pasos medios hasta el objetivo y tiempo de ejecución es: γ : 0.95, α : 0.05, ϵ : 0.5, ϵ_{decay} : 0.001, ϵ_{end} : 0.01, `num_train_episodes`: 4000, `rewards`: (finish: 100, fall:-100, step:-1)

Observamos que ciertos hiperparámetros no concuerdan con sus respectivos máximos en cada análisis individual, esto es debido al muestreo aleatorio. Es altamente probable que una ejecución realizada con los parámetros observados como óptimos dentro del espacio discreto de hiperparámetros mostrase mejores resultados.

Se recomienda una exploración prolongada y cuidadosa para entornos estocásticos como este. La robustez de la política aprendida frente a acciones no deterministas es clave, y esto solo se logra si el agente experimenta y aprende de una amplia variedad de trayectorias, incluyendo aquellas donde la "mala suerte" lo desvía de su curso intencionado.

5. REINFORCE

El algoritmo *REINFORCE* utiliza gradiente de política. Su objetivo es ajustar los parámetros de una política para maximizar la recompensa esperada. A diferencia de los métodos basados en valor, *REINFORCE* aprende directamente la política sin intentar modelar el entorno.

5.1. Funcionamiento del Algoritmo

En su forma básica, el algoritmo REINFORCE actualiza los parámetros de la política en la dirección que aumenta la probabilidad de tomar acciones que condujeron a altas recompensas. Esto se logra utilizando el gradiente de la recompensa esperada con respecto a los parámetros de la política.

5.1.1. Parámetros

Los parámetros para la primera implementación del algoritmo REINFORCE son:

1. **Gamma (γ):** Factor de descuento para las recompensas futuras. Un valor entre 0 y 1. Determina la importancia de las recompensas futuras en comparación con las inmediatas.
2. **Learning Rate (α):** Tasa de aprendizaje para la actualización de los parámetros de la política. Controla el tamaño de los pasos en la dirección del gradiente.
3. **Learning Rate Decay (lr_decay):** Factor por el cual se multiplica la tasa de aprendizaje después de cada episodio. Permite reducir la tasa de aprendizaje a medida que avanza el entrenamiento.
4. **Training Episodes:** Número total de episodios durante los cuales el agente aprenderá.
5. **T_MAX:** Número máximo de pasos permitidos por episodio durante el entrenamiento y la evaluación.

Adicionalmente, como en el resto de algoritmos, modificamos las recompensas del entorno:

1. **Finish Reward:** Recompensa obtenida al alcanzar el estado objetivo.
2. **Fall Reward:** Penalización recibida al caer por el acantilado.

3. **Step Reward:** Recompensa (o penalización) recibida por cada paso que no resulta en terminar el episodio.

5.1.2. El algoritmo por partes

Los pasos principales del algoritmo REINFORCE implementado son:

1. **Inicialización:**

- a) Se inicializa la tabla de política (θ) de manera que, para cada estado, todas las acciones tengan la misma probabilidad $P(a|s) = 1/|A(s)|$.
- b) Se establecen los hiperparámetros: γ , α , 'lr_decay', 'TRAINING_EPISODES'.

2. **Bucle de Entrenamiento** (repetir para cada episodio TRAINING_EPISODES veces):

a) **Generar trayectoria:**

- El agente interactúa con el entorno siguiendo la política actual $\pi(a|s; \theta)$ para generar una secuencia de estados, acciones y recompensas: $s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T$.

b) **Calcular retornos descontados:**

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_{k+1} \quad \forall t \in [0, T-1]$$

c) **Actualizar política:**

- Para cada (s_t, a_t) en el episodio:

$$\begin{aligned} \pi(a_t|s_t) &\leftarrow \pi(a_t|s_t) + \alpha G_t (1 - \pi(a_t|s_t)) \\ \pi(a'|s_t) &\leftarrow \pi(a'|s_t) - \alpha G_t \pi(a'|s_t) \quad \forall a' \neq a_t \\ \text{Renormalizar: } \pi(\cdot|s_t) &\leftarrow \frac{\pi(\cdot|s_t)}{\sum_a \pi(a|s_t)} \end{aligned}$$

- Pérdida del episodio:

$$L = -\frac{1}{T} \sum_{t=0}^{T-1} G_t \log \pi(a_t|s_t)$$

d) **Ajustar tasa de aprendizaje:**

$$\alpha \leftarrow \alpha \cdot \text{lr_decay}$$

3. **Fin del entrenamiento:** La política aprendida se aprende de la tabla de posibilidades final. Para cada estado, se escoge la acción con más probabilidad como la política a seguir.

5.2. Estudio

En este apartado se analizarán los resultados obtenidos al experimentar con diferentes configuraciones de hiperparámetros para la versión original del algoritmo REINFORCE en el entorno ‘CliffWalking-v0’.

5.2.1. Toma de muestras

Se utilizó un diseño factorial fraccionado para explorar el espacio de parámetros. Se seleccionaron los siguientes rangos para los hiperparámetros:

$$\begin{aligned}
 \textit{Gamma} &\in \{0.9, 0.95, 0.99\}, \\
 \textit{Training Episodes} &\in \{1000, 10000, 50000\}, \\
 \textit{Finish Reward} &\in \{0, 10, 100\}, \\
 \textit{Fall Reward} &\in \{-10, -100, -1000\}, \\
 \textit{Step Reward} &\in \{0, -1, -10\}, \\
 \textit{Learning Rate} &\in \{0.001, 0.01, 0.1, 0.5, 1.0\}, \\
 \textit{Learning Rate Decay} &\in \{0.9, 0.95, 0.99, 0.999, 1.0\}
 \end{aligned}$$

De un total de $3^5 \times 5^2 = 243 \times 25 = 6075$ combinaciones posibles, se generaron y evaluaron 250 combinaciones mediante el método ‘create_fractional_factorial_design’. La métrica principal para evaluar el rendimiento es el promedio de pasos (‘avg_steps’) necesarios para completar un episodio en 100 episodios de evaluación, donde un menor número de pasos es mejor.

5.2.2. Resultados

A continuación, se presentan los diagramas de caja (boxplots) que muestran el efecto de cada hiperparámetro en el promedio de pasos.

5.2.2.1 Importancia relativa de los hiperparámetros

Se definieron rangos y valores discretos para cada hiperparámetro a estudiar. A partir de estos, se generaron 250 combinaciones de hiperparámetros de forma aleatoria para los

experimentos. Los rangos considerados fueron:

- γ : {0.9, 0.95, 0.99}
- **Learning rate**: {0.001, 0.01, 0.1, 0.5, 1.0}
- **Learning rate decay**: {0.9, 0.95, 0.99, 0.999, 1.0}
- **Número de Episodios de Entrenamiento**: {1000, 10000, 50000}
- **Señal de Recompensa (finish)**: {0, 10, 100}
- **Señal de Recompensa (fall)**: {10, -100, -1000}
- **Señal de Recompensa (step)**: {0, -1, -10}
- t_{max} : {200}

Tras la ejecución de las 250 combinaciones de hiperparámetros, se observó una amplia gama de rendimientos. El análisis de importancia de características mediante un modelo *Random Forest* (OOB Score: 0.5007) indicó que los parámetros más influyentes en el número de pasos en evaluación para episodios exitosos fueron, en orden descendente:

- **step reward**: 0.3730
- **Learning rate decay**: 0.2126
- γ : 0.1157
- **Learning rate**: 0.0937
- **Número de Episodios de Entrenamiento**: 0.0908
- **fall reward**: 0.0691
- **finish reward**: 0.0452

En este caso la importancia de la recompensa por paso es mayor a la del resto de parámetros, sin embargo, al tener un OOB score tan bajo, estos ratios es probable que no reflejen la realidad.

5.2.2.2 Efecto de Finish Reward

En la figura 31 podemos ver que la recompensa de finalización del entorno no influye demasiado en la media de pasos para valores mayores de 0. Para 0 vemos que los resultados no son demasiado prometedores.

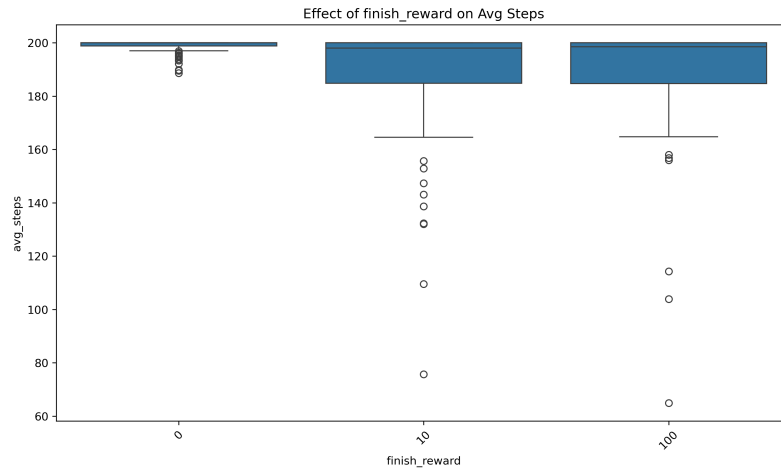


Figura 31: Promedio de pasos respecto a la recompensa de finalización

En la figura 32 podemos ver que la recompensa de finalización, al igual que en el gráfico anterior, no influye demasiado, en el sentido que tiene una tasa de éxito similar siempre que sea mayor que 0.

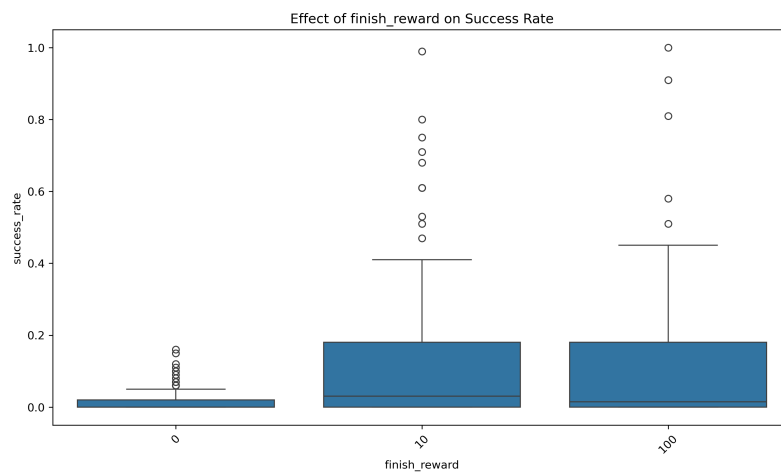


Figura 32: Ejecuciones exitosas respecto a la recompensa de finalización

En la figura 33 podemos ver que la recompensa de finalización reduce algo el tiempo de entrenamiento con una recompensa de 100 respecto a una de 10 o 0.

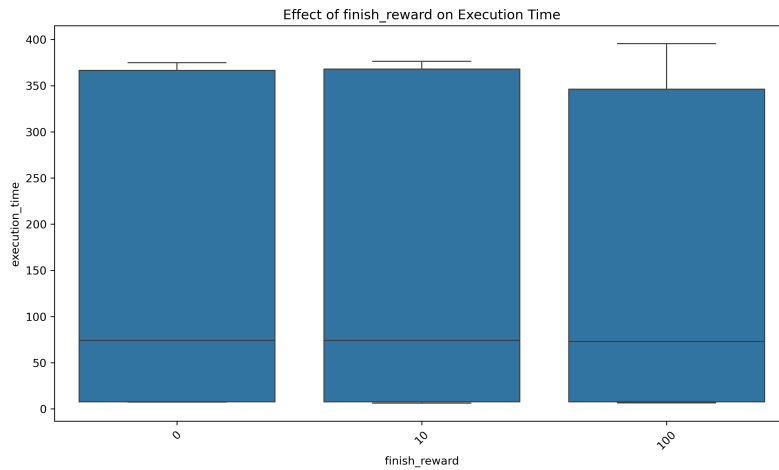


Figura 33: Tiempo de entrenamiento respecto a la recompensa de finalización

5.2.2.3 Efecto de Fall Reward

En la figura 34 podemos ver que la recompensa de caída del entorno no influye demasiado en la media de pasos. La mayoría de ejecuciones tiene una media de pasos cercana o igual a T_MAX, distribuidas de manera bastante equilibrada entre los tres posibles valores de recompensa estudiados.

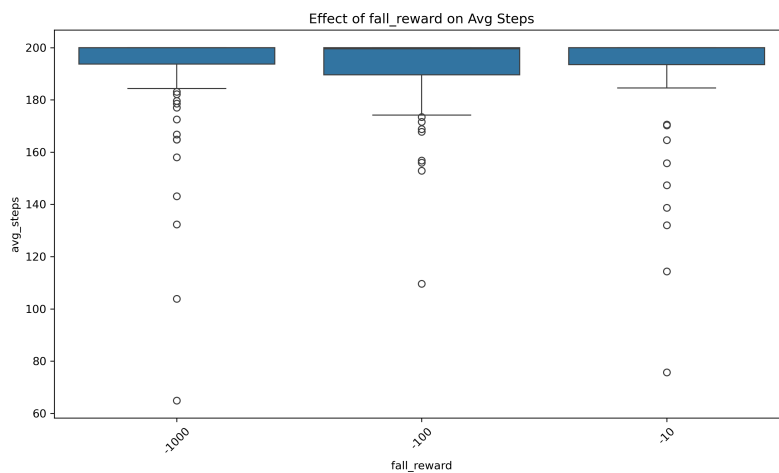


Figura 34: Promedio de pasos respecto a la recompensa de caída

En la figura 35 podemos ver que la recompensa de caída no influye demasiado en la tasa éxito, de todas maneras, se puede apreciar una mayor estabilidad con una recompensa de -100 sobre las demás.

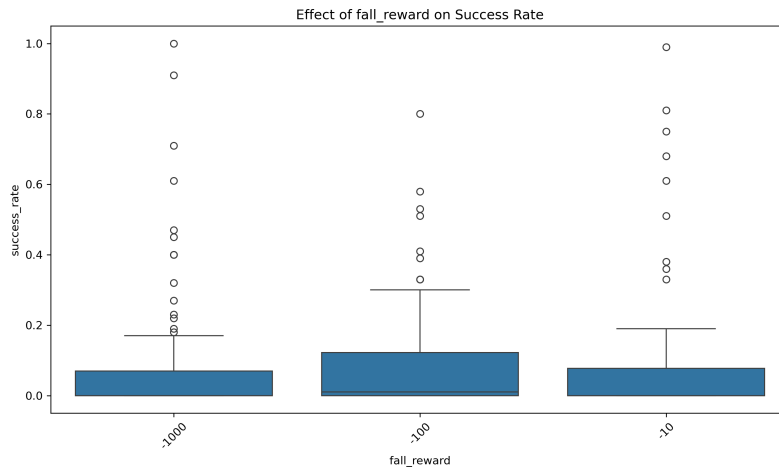


Figura 35: Ejecuciones exitosas respecto a la recompensa de caída

En la figura 36 podemos ver que la recompensa de caída del entorno no tiene ningún impacto significativo sobre el tiempo de entrenamiento del agente.

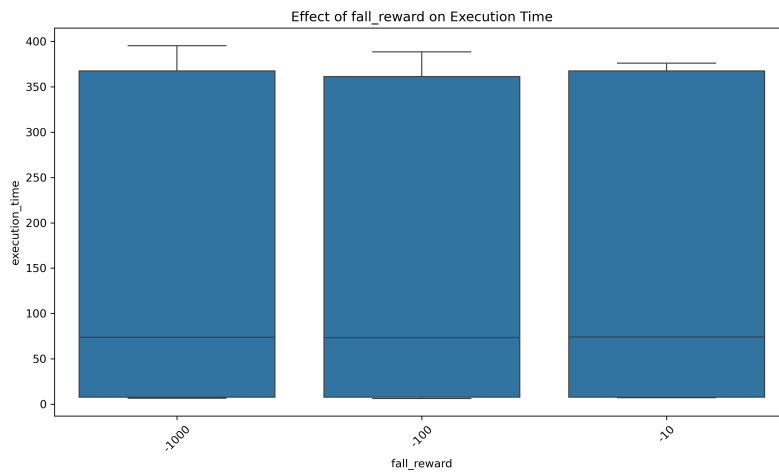


Figura 36: Tiempo de entrenamiento respecto a la recompensa de caída

5.2.2.4 Efecto de Step Reward

En la figura 37 podemos ver que la recompensa por paso del entorno es uno de los parámetros que tiene una importancia relativamente alta a la hora de reducir la cantidad de pasos promedio, siendo una recompensa por paso de 0 el mejor valor para reducir el promedio de pasos. Este resultado difiere del model_based donde una recompensa de 0 resulta en los peores resultados.

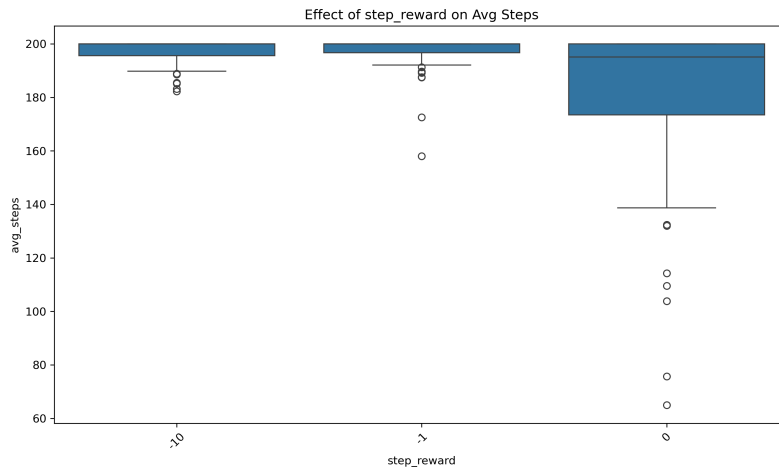


Figura 37: Promedio de pasos respecto a la recompensa por paso

En la figura 38 podemos ver que la recompensa por paso del entorno es uno de los parámetros que tiene una importancia relativamente alta a la hora de conseguir políticas que resultan en ejecuciones exitosas, siendo una recompensa por paso de 0 el mejor valor para este parámetro. Este resultado es consistente en relación al gráfico de la figura anterior y sigue siendo contrario al del `model_based`, donde una recompensa de 0 resulta en los peores resultados.

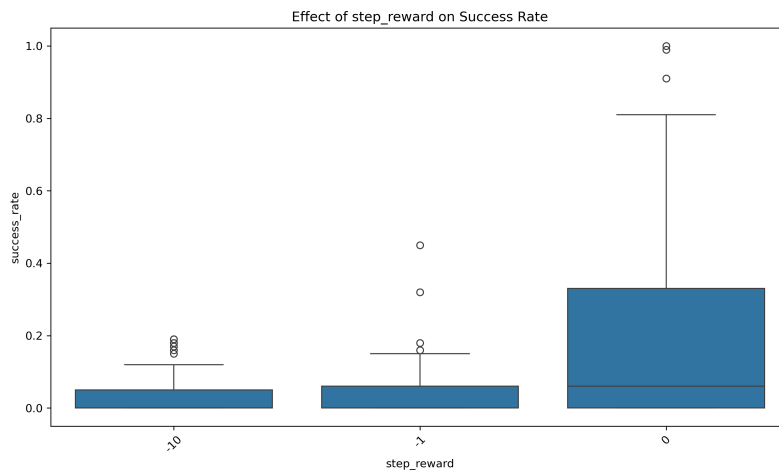


Figura 38: Ejecuciones exitosas respecto a la recompensa por paso

En la figura 39 podemos ver que la recompensa por paso del entorno no tiene ningún impacto significativo sobre el tiempo de entrenamiento del agente.

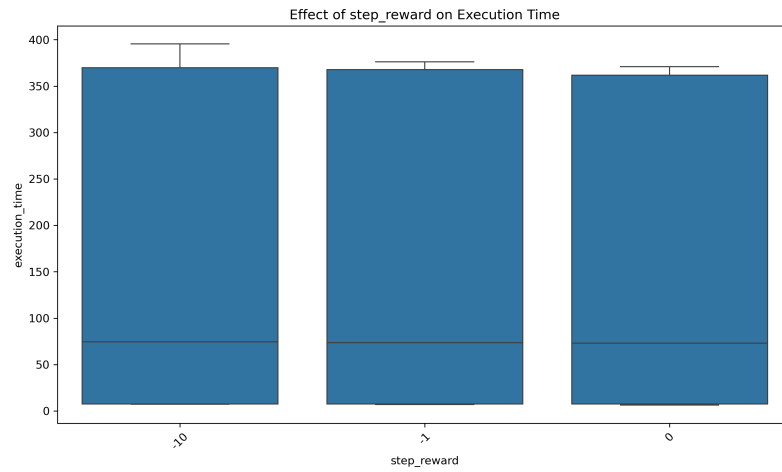


Figura 39: Tiempo de entrenamiento respecto a la recompensa por paso

Análisis esperado: Una recompensa por paso negativa incentiva soluciones más cortas. Una recompensa de cero o positiva podría llevar a que el agente no tenga prisa por terminar.

5.2.2.5 Efecto de Gamma (γ)

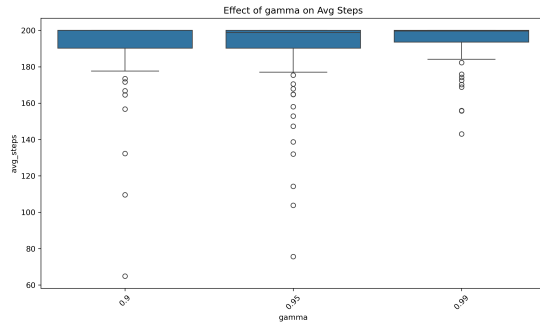


Figura 40: Pasos promedio en función de las recompensas (todas las ejecuciones)

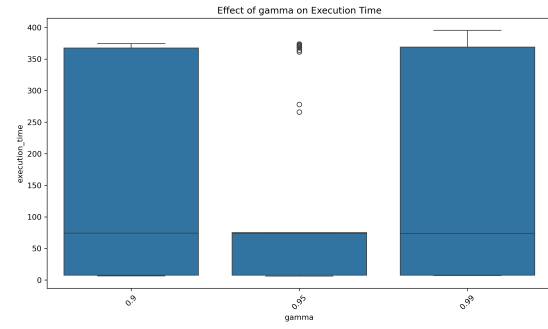


Figura 41: Pasos promedio en función de las recompensas (ejecuciones exitosas)

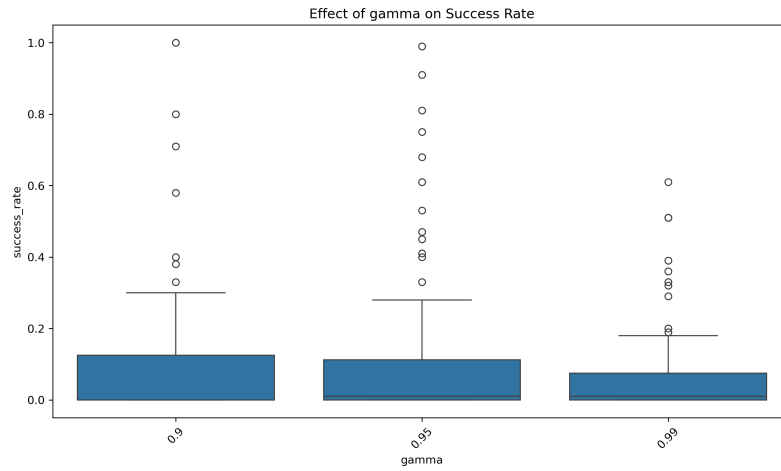


Figura 42: Efecto de Gamma en el promedio de pasos.

Se puede observar en los *BoxPlots* anteriores como los distintos valores de *Gamma* no tienen un efecto destacable sobre los resultados en términos de número de pasos y tasa de éxito. Sin embargo, el tiempo de ejecución con una *Gamma* de 0.95 se reduce notablemente, sin tener tampoco un efecto demasiado grande sobre el resto de parámetros.

5.2.2.6 Efecto de Training Episodes

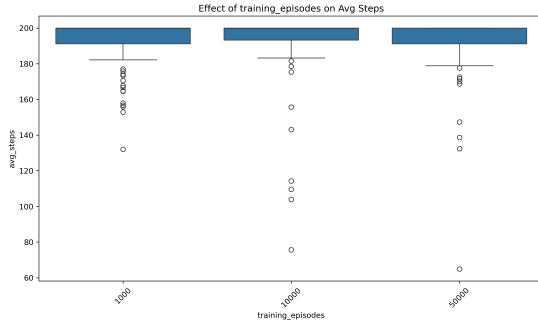


Figura 43: Pasos promedio en función de las recompensas (todas las ejecuciones)

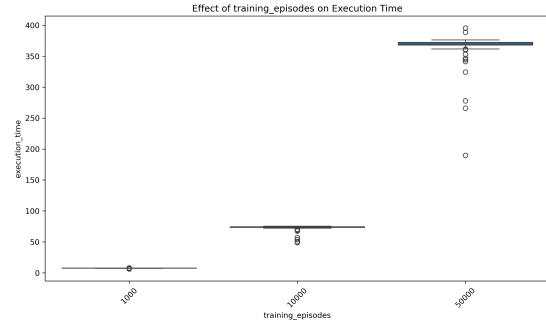


Figura 44: Pasos promedio en función de las recompensas (ejecuciones exitosas)

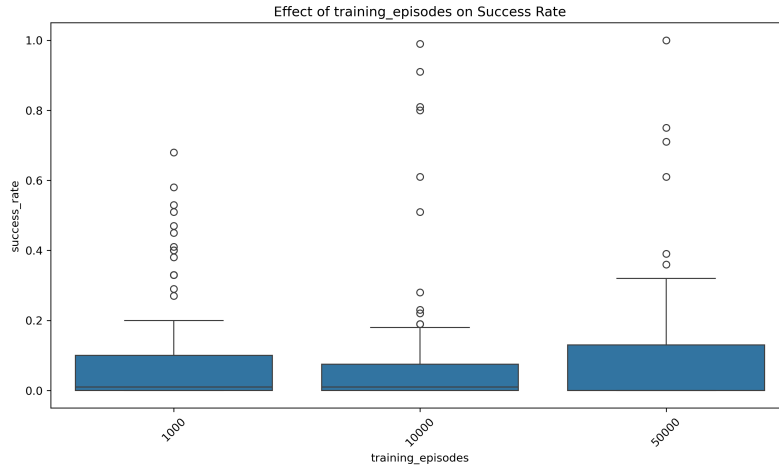


Figura 45: Efecto de Gamma en el promedio de pasos.

Sorprendentemente, el efecto de los episodios de entrenamiento sobre la tasa de éxito y el numero de pasos es mínimo. Todos los valores tienen resultados muy similares. El único efecto es por supuesto el tiempo de ejecución, que aumenta proporcionalmente al numero de episodios por motivos obvios.

5.2.2.7 Efecto de Learning Rate (α)

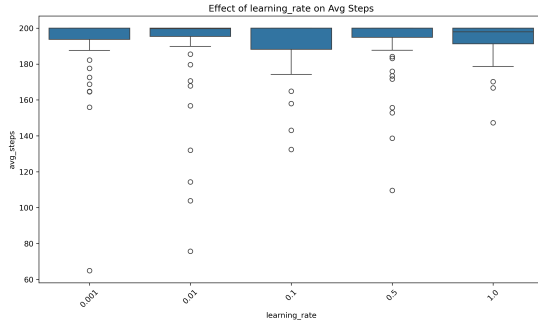


Figura 46: Pasos promedio en función de las recompensas (todas las ejecuciones)

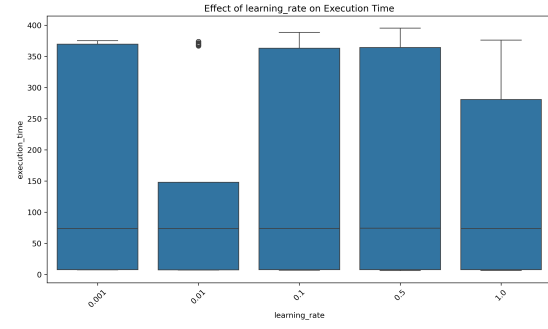


Figura 47: Pasos promedio en función de las recompensas (ejecuciones exitosas)

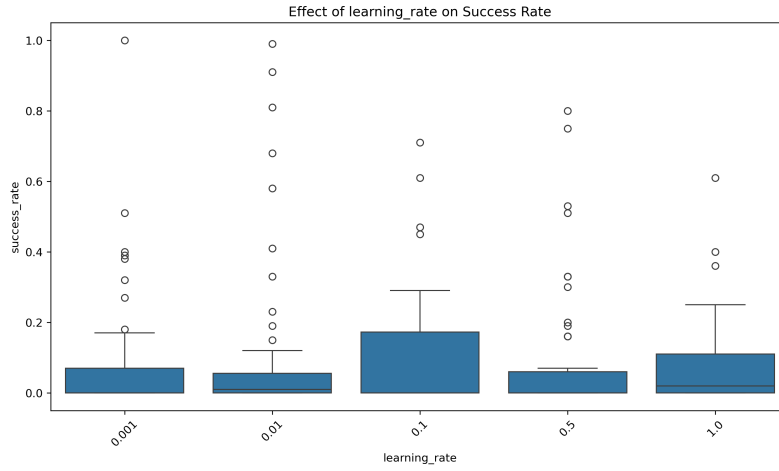


Figura 48: Efecto de Gamma en el promedio de pasos.

De nuevo, los resultados se asemejan a los parámetros anteriores. Aunque en este caso podemos comprobar que una tasa de aprendizaje de 0.1 proporciona una tasa de éxito y numero de pasos ligeramente mejor.

En cuanto al tiempo, son todos bastantes similares, excepto por la asignación de 0.01 que tiene un tiempo de ejecución mucho menor, y la de 1.0, que es ligeramente más rápida. Una observación es que el aumento del tiempo no parece ser proporcional a los cambios de las learning rates, como se puede observar en la figura.

5.2.2.8 Efecto de Learning Rate Decay

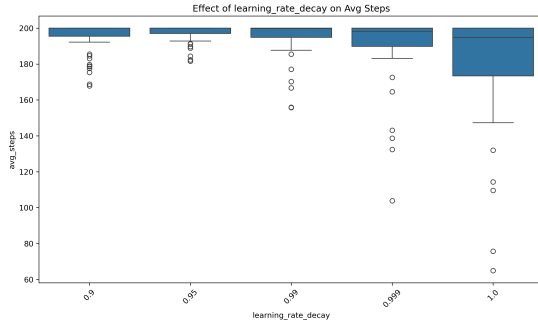


Figura 49: Pasos promedio en función de las recompensas (todas las ejecuciones)

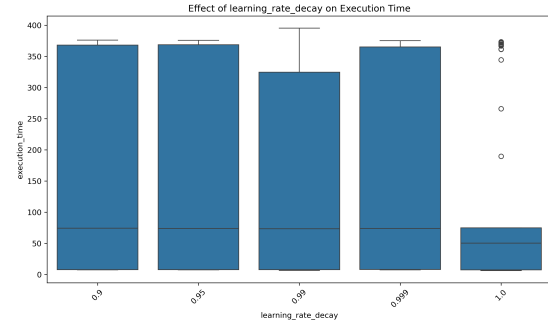


Figura 50: Pasos promedio en función de las recompensas (ejecuciones exitosas)

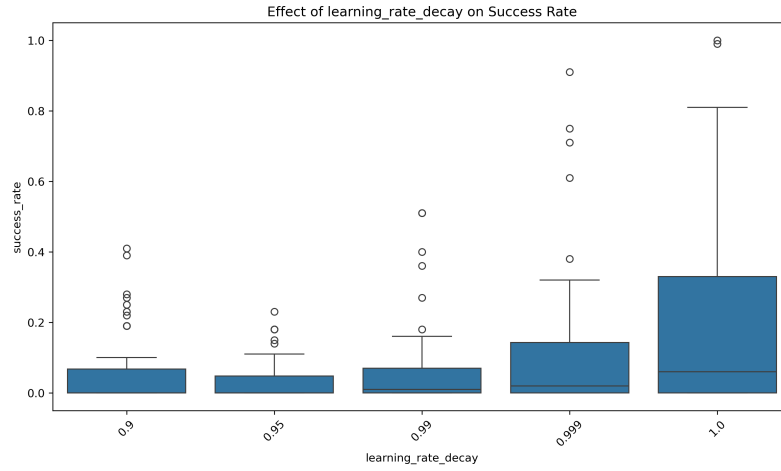


Figura 51: Efecto de Gamma en el promedio de pasos.

Estas figuras son de las mas prometedoras en el estudio de este algoritmo, podemos comprobar que un *learning rate decay* de 1.0 es mejor en todos los aspectos. Mejora respecto al resto de valores en términos de numero de pasos y tasa de éxito. Y el tiempo de ejecución es exageradamente más rápido.

5.2.2.9 Mejores configuraciones encontradas

Para posicionar una configuración por encima de otra, hacemos una ponderación teniendo en cuenta los pasos promedios resultantes de aplicar la política encontrada con dicha configuración, el tiempo de entrenamiento y la tasa de ejecuciones exitosas. Teniendo esto en cuenta, vemos que la mejor configuración, pese a tener unos 10 pasos más de media con respecto a la segunda mejor debido a que tarda unas 3-4 veces menos.

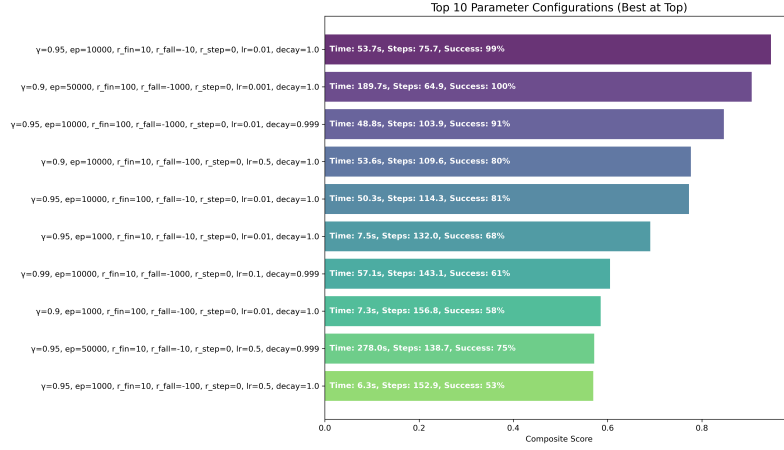


Figura 52: Top 10 configuraciones REINFORCE

5.3. Intento de mejora

Los resultados obtenidos en el anterior estudio muestran que la implementación del algoritmo, por lo menos con los parámetros seleccionados, no es nada efectiva. Para intentar mejorar esto, se implementó una variación del algoritmo *REINFORCE*, con algunas mejoras. La principal de estas mejoras siendo que se añadió un **Bono de entropía regulado** al algoritmo para evitar que cayese en mínimos locales, fomentando la exploración. Ya que se pensaba que el problema del mal rendimiento venía arraigado por la caída en mínimos locales.

Sin embargo, tras una toma de 150 muestras, añadiendo $entropy_bonus \in \{0.01, 0.1, 0.2\}$, se concluyó que no había supuesto ningún tipo de mejora. Se puede observar en la siguiente figura:

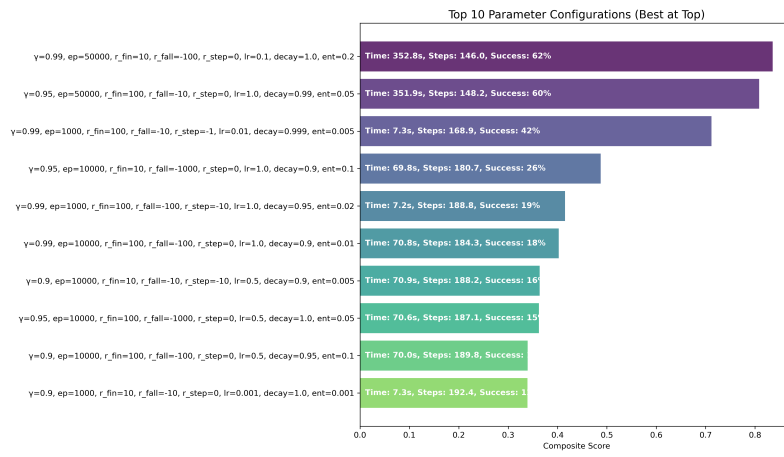


Figura 53: Top 10 configuraciones del intento de mejora

6. Bibliografía

Referencias

- [1] James Bergstra and Yoshua Bengio. *Random Search for Hyper-Parameter Optimization*. Journal of Machine Learning Research, 13(Feb):281–305, 2012. Disponible [aquí](#).
- [2] Farama Foundation. *Cliff Walking Environment Documentation*. Gymnasium, 2023. Disponible [aquí](#).
- [3] Wikipedia. *Random Forest* Disponible [aquí](#).
- [4] Arthur Juliani. *Maximum Entropy Policies in Reinforcement Learning & Everyday Life*. Medium, 2017. Disponible [aquí](#).